Natural Language Processing Final Project

1 Overview

In this project, each group will design and implement a retrieval-augmented generation (RAG) system using the open-source MultiHop-RAG benchmark. The system must retrieve evidence from a knowledge base and use it to answer multi-hop questions. The project consists of knowledge-base processing, RAG system implementation, and evaluation on the complete benchmark.

Your report must explicitly state that the system uses RAG, explain the method, and provide retrieval and generation records. The project grade combines Report Quality, Workload, Response Accuracy, Code, and Novelty. Response Accuracy accounts for 40% of the total. Official overall Accuracy of at least 65%, 70%, and 75% corresponds to Pass, Good, and Excellent within this component. See Section 4 for grading and Section 5 for submission requirements.

2 RAG System Implementation

2.1 Task Description

The input consists of a question and the knowledge base; the output is a short answer. The system may combine facts from multiple documents to make a comparison or inference. For questions that cannot be answered from the evidence, output `Insufficient Information`. Save final answers in a JSON file indexed by question ID.

2.2 RAG Method

RAG combines external knowledge retrieval with language-model generation. Explain how your system processes the knowledge base, retrieves and selects evidence, builds the context, and generates an answer. You may choose your own chunking, sparse or dense retrieval, reranking, query rewriting, and multi-round retrieval or generation methods.

Model size limit: Each individual model must have no more than **10 billion total parameters (10B)**. For mixture-of-experts models, count all parameters, not only active parameters. Quantization does not change the parameter count. Training or fine-tuning is optional, is not required to complete the project, and does not earn points by itself. You may use frozen models and improve the system through retrieval methods, prompts, and hyperparameter tuning. Model choice, training or fine-tuning, prompts, sampling settings, reasoning mode, context length, output length, retrieval depth, and number of model calls are otherwise unrestricted. Document the settings and resource usage.

2.3 Required System Features

- **Evidence retrieval and answering:** Implement the complete question-to-evidence-to-answer workflow. The RAG method described in the report must match the system used to produce predictions.
- **Traceable execution:** Save retrieved evidence, document sources, the context supplied to the generator, and the final answer for each question. Multi-round systems must retain records for each retrieval and generation round.
- **Batch processing and output validation:** Process the complete question set, export valid JSON, log failures and retries, and provide reproducible execution commands.

2.4 Evaluation Data and Question Types

Use the complete official corpus of 609 English documents and all 2,556 questions, covering inference, comparison, temporal, and unanswerable queries. All questions have equal weight. Do not reduce the corpus or final evaluation set. The original `answer` and `evidence_list` fields are for evaluation and analysis only; do not use them as system inputs, indexed content, training labels, or answer lookup sources.

Public models, frameworks, and tools are allowed with attribution. Disclose the sources and purposes of any additional resources. Reference answers or annotated reference evidence must not replace actual retrieval.

3 Evaluation

Evaluation consists of system debugging, prediction scoring on the complete benchmark, and review of reports and code. You may use subsets for debugging; final scoring requires a prediction JSON file covering all 2,556 questions. The instructor will use the official MultiHop-RAG `qa_evaluate.py` script for the benchmark score and separately assess Report Quality, Workload, Code, and Novelty, while verifying the actual RAG workflow.

Submit `predictions.json` as a UTF-8 JSON array. Question IDs are fixed as `q0000` through `q2555` in the order of the official question file. Each record must contain `question_id` and `model_answer`:

```
[
  {"question_id": "q0000", "model_answer": "predicted short answer"},
  {"question_id": "q0001", "model_answer": "Insufficient Information"}
]
```

This example illustrates the format only. The final file must cover every question without duplicate or altered IDs and must not contain reference answers. Store only the final short answer in `model_answer`; keep retrieval traces and reasoning explanations in separate execution records.

- **Scoring rule: Benchmark score B = 100 × official overall Accuracy.** Use the fixed official script version distributed with the assignment and apply its rules unchanged. Section 4 maps B to a score band for the Response Accuracy component, which contributes 40% of the project grade.

The official script lowercases answers and counts a question as successful when the prediction and reference share at least one whitespace-separated token. Its reported Precision, Recall, F1, and Accuracy have the same value in this version. Use overall Accuracy for grading and label reported metrics as official-script results.

- **Submission preparation:** The course utility validates IDs, maps them to official queries and question types, and supplies empty answers for missing or failed predictions. All 2,556 questions remain in the denominator. Reference answers are taken from the official dataset.
- **Evaluation metrics:** Report the official overall Accuracy, the corresponding system score, and official results by question type. Retrieval metrics may be added where useful; specify the script and calculation used.
- **Resources and analysis:** Report model names and parameter counts, hardware, model-call counts, token usage, and runtime. Analyze representative successes and failures. Experimental settings are chosen by each group and must be disclosed to support interpretation and reproduction.

Submit the final prediction file `predictions.json`.

3.1 Optional Reference Codebases

- **RAG From Scratch: start here:** LangChain notebooks explain indexing, retrieval, and generation. Begin with parts 1-4; adapt the example model interfaces.

https://github.com/langchain-ai/rag-from-scratch

- **RAG Techniques: method examples:** Consult Basic RAG, Optimizing Chunk Sizes, Query Transformations, and Reranking for implementation ideas.

https://github.com/NirDiamant/RAG_Techniques

- **Ollama PDF RAG: local demo:** A local PDF chat example. Adapt its document input to the benchmark JSON corpus and add batch prediction export.

https://github.com/tonykipkemboi/ollama_pdf_rag

- **MultiHop-RAG: benchmark integration:** See simple_retrieval.py and qa_llama.py for retrieval and answering; follow repository dependency instructions.

https://github.com/yixuantt/MultiHop-RAG

All references are optional. Keep models within 10B; export required JSON and logs; use the fixed official evaluator. Cite code, versions, and modifications. Reference use alone earns no credit.

4 Grading Breakdown

4.1 Components and Weights

- **Report Quality (20%):** Clarity, completeness, and professionalism of the written report.
- **Workload (20%):** Work involved in knowledge-base processing, data preparation, and RAG system and model development.
- **Response Accuracy (40%):** Validity and accuracy of the system's JSON predictions, assessed according to Section 4.2.
- **Code (10%):** Organization, readability, reproducibility, and efficiency of the code.
- **Novelty (10%):** Innovativeness of the system implementation, knowledge-base processing, and data preparation methods.

Each component is scored from 0 to 100 before weighting. Let R denote Report Quality, W Workload, A Response Accuracy, C Code, and N Novelty. The total project grade is:

$$G = 0.20R + 0.20W + 0.40A + 0.10C + 0.10N$$

4.2 Response Accuracy Component

After verifying the RAG workflow and prediction file, use the unrounded benchmark score B to determine the score band for component A. Displayed scores may use two decimal places. The instructor assigns the specific score within the band. This table applies only to Response Accuracy, worth 40% of the total project grade.

Component rating	**Benchmark score B**	**Component A score band**
Excellent	75 or above	90-100
Good	70 or above, below 75	80-89
Pass	65 or above, below 70	60-79
Below Pass	Below 65	Below 60

For example, B = 79.26 places A in the 90-100 band, contributing 36-40 points to the project grade. The other four components determine the remaining 60 points. Reaching a benchmark threshold alone does not complete the assignment without a declared and demonstrated RAG workflow. The instructor may inspect source documents, contexts, and predictions, rerun the system, and ask members to explain their contributions.

5 Submission

Follow the course group arrangements. Before **November 1, 2026**, the group leader must submit `GroupXX_NLP_RAG.zip` containing the materials below. Submit source code through a GitHub repository; do not include it in the archive. The report must include every member's name, student ID, specific contributions, and the GitHub repository URL. All materials except source code must be submitted by this deadline. The submission platform will be announced through the course.

- **Project report:** `GroupXX_project_report.docx` and an identical PDF, including a cover page, member information, the GitHub repository URL, methods, results, and analysis.
- **Predictions and scores:** `predictions.json` and scoring-script outputs covering all questions, with overall and per-type results.
- **Execution records:** Question-linked retrieval results, actual contexts, and answers, plus model versions, parameter counts, settings, resource usage, failures, and retries.

Upload source code, configuration files, dependency information, and a README to GitHub. Include the full repository URL in the report and ensure that the instructor can access it. The README must provide commands for prediction and scoring. Repository contents must match the submitted predictions and report; model weights need not be uploaded.

Use the provided report template. The entire report, including the cover page, must not exceed **10 pages**. Organize the main text into the following six sections and cite all models, datasets, papers, and reused code.

- **Introduction:** Task background, research questions, and project objectives.
- **Methodology:** Explicitly declare the use of RAG. Describe the complete method, model sizes, and experimental settings; distinguish your implementation from reused components.
- **Evaluation Result:** Benchmark score, Response Accuracy component band, supplementary metrics, results by question type, and resource usage on the complete question set.
- **Analysis:** Use source evidence to analyze representative cases, explain performance and failures, and discuss limitations.
- **Contributions:** Identify each member's implementation, experimental, analytical, and documentation work.
- **Conclusion:** Summarize the main findings and directions for improvement.

Before submission, check filenames, the accessibility of the GitHub URL in the report, executable repository code, prediction completeness, and accurate reporting of settings and results. Describe how the public benchmark was used for debugging and parameter selection; do not characterize these results as hidden-test performance. Course rules on academic integrity and tool use apply.

Official resources. MultiHop-RAG by Yixuan Tang and Yi Yang. Data license: ODC-BY.

Repository: https://github.com/yixuantt/MultiHop-RAG
Dataset: https://huggingface.co/datasets/yixuantt/MultiHopRAG
Paper: https://arxiv.org/abs/2401.15391